QueryAPIGate HOW-TO GUIDE 39 · OPERATE

Read the example APIs as a template for your own

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=39-walk-through-the-example-apis

Time: 20 minutes. **You'll end up with:** an understanding of *why* each of the 11 example queries, 4 collections and 5 roles is set up the way it is - and a recipe to copy for each kind of API you'll build.

Load them first: Try it without any setup. What each scenario *does* is in EXAMPLES.md; this guide is about the choices behind them. Every definition below is read from a loaded server.

The shape: one collection and one role per audience

AUDIENCE	COLLECTION	ROLE	RATE LIMIT
Internal reporting	`examples-reporting` (3 queries)	`example-reporting`	`200/hour`
Wall dashboards	`examples-dashboard` (4 queries)	`example-dashboard`	`600/minute`
Bulk exports	`examples-export` (2 queries)	`example-export`	`20/hour`
An external partner	`examples-partner` (2 queries)	`example-partner`	`60/minute`
Executives (reporting + dashboards)	both of the first two	`example-executive`	`300/hour`

Every role has `connections: []` and no writes. That's the first thing to copy: **no role gets a connection grant**, so no key made from them can run ad-hoc SQL or browse a schema - only the saved queries in its collections. And because the grant is a *collection*, adding a query to `examples-reporting` later reaches every reporting key at once (guide 7).

The rate limits follow the traffic each audience should produce: dashboards poll often (`600/minute`), an export should run a few times a day (`20/hour`). A limit is cheapest to set before the first caller depends on there being none (guide 17).

`example-executive` shows that a role can span collections - two audiences' queries, one key - without copying any query.

Reporting: parameters with safe defaults

```sql
-- example_monthly_revenue        months: integer, 1-24, default 6
SELECT strftime('%Y-%m', rental_date) AS month, COUNT(*) AS rentals, ROUND(SUM(amount), 2) AS revenue
FROM rental WHERE rental_date ≥ date('now', '-' || :months || ' months')
GROUP BY month ORDER BY month DESC
```

- **Every parameter has a default and bounds.** Called with no parameters, it answers something sensible; called with `months=1000`, it's refused before the database is touched. The bound is the cost control: nobody scans 80 years.
- **A bound parameter inside an expression** - `'-' || :months || ' months'` - is still a bound value, never SQL text. You don't need `{months}` text substitution for this (guide 6).

```sql
-- example_top_films        top_n: integer 1-50, default 10;  category: one of six, optional
SELECT * FROM (SELECT f.title, f.category, f.rating, COUNT(*) AS rentals,
                      RANK() OVER (ORDER BY COUNT(*) DESC) AS rank
               FROM rental r JOIN film f ON f.film_id = r.film_id
               WHERE (:category IS NULL OR f.category = :category)
               GROUP BY f.film_id)
WHERE rank ⩽ :top_n ORDER BY rank, title
```

- **An optional filter:** `(:category IS NULL OR f.category = :category)`. An optional parameter with no value binds as `NULL`, so one query serves "all categories" and "one category".
- **`enum` for a closed set** - the six categories. A typo gets a list of valid values back, not an empty result.

Dashboard: no parameters, cached

```sql
-- example_kpi_active_rentals        cache_ttl: 30
SELECT COUNT(*) AS active_rentals FROM rental WHERE return_date IS NULL
```

All four dashboard queries carry `cache_ttl: 30`. Ten screens polling every five seconds cost one database query every 30 seconds, not 120 a minute. The trade - numbers up to 30 seconds old - is right for a wall display and wrong for anything a person just changed (guide 8). `example_recent_rentals` takes an `hours` window (1-168, default 24); each distinct value is cached separately.

Export: stream, and make it incremental

- `example_all_rentals` - no parameters, about 20,000 rows over six joined tables. Meant to be called with `?stream=true&format=csv` (guide 9) or `queryapigate export` (guide 30).
- `example_rentals_since` - the same idea, incremental: `since` (a `YYYY-MM-DD` string, checked by `pattern`, default `2000-01-01`) and `ORDER BY r.rental_id`. A nightly job asks only for what's new; a stable order makes the files comparable from one run to the next.

The low `20/hour` limit says: this is a batch job, not something to poll.

Partner: required, strict, minimal

```text
example_film_lookup     film_id: integer, required, ⩾ 1
example_film_search     text: string, required, 2-30 characters, pattern [A-Za-z ]+
```

- **Required, no defaults.** A partner's call should say exactly what it wants.

- **`pattern` closes a hole `LIKE` opens.** `example_film_search` runs `WHERE title LIKE '%' || :text || '%'`. The value is bound, so it can't inject SQL - but `%` and `_` are wildcards *inside* `LIKE`. The pattern allows only letters and spaces, so a partner can't turn a search into "give me everything". Verified: `text=%%` is refused - *text must match the pattern [A-Za-z]+* - and `text=gar` returns the films with "gar" in the title.
- **Small results.** A lookup returns one film's figures, not rows from the rentals behind them.

For a real partner, add an expiry to the key itself - `{"name": "acme-corp", "role": "example-partner", "expires_at": "2026-12-31"}` - and consider `allowed_ips` (guides 18 and 19).

Copying a scenario for your own data

1. Create the collection by saving your first query into it - `"collection": "sales-reporting"`.
2. Create the role: `{"name": "sales-reporting", "connections": [], "collections": ["sales-reporting"], "rate_limit": "200/hour"}`.
3. Create keys from the role, one per team or integration (guide 15).
4. Check the result as the key would see it: `GET /catalog` with its secret (guide 21).

Then remove the examples - `queryapigate examples unload` removes exactly what they installed, nothing of yours.

Next steps

- Use bound parameters safely - every rule used above.
- Group queries into a collection.
- Create a role.